Generation in three parts: the LLM calls

Nov 3, 2026 · @Robin

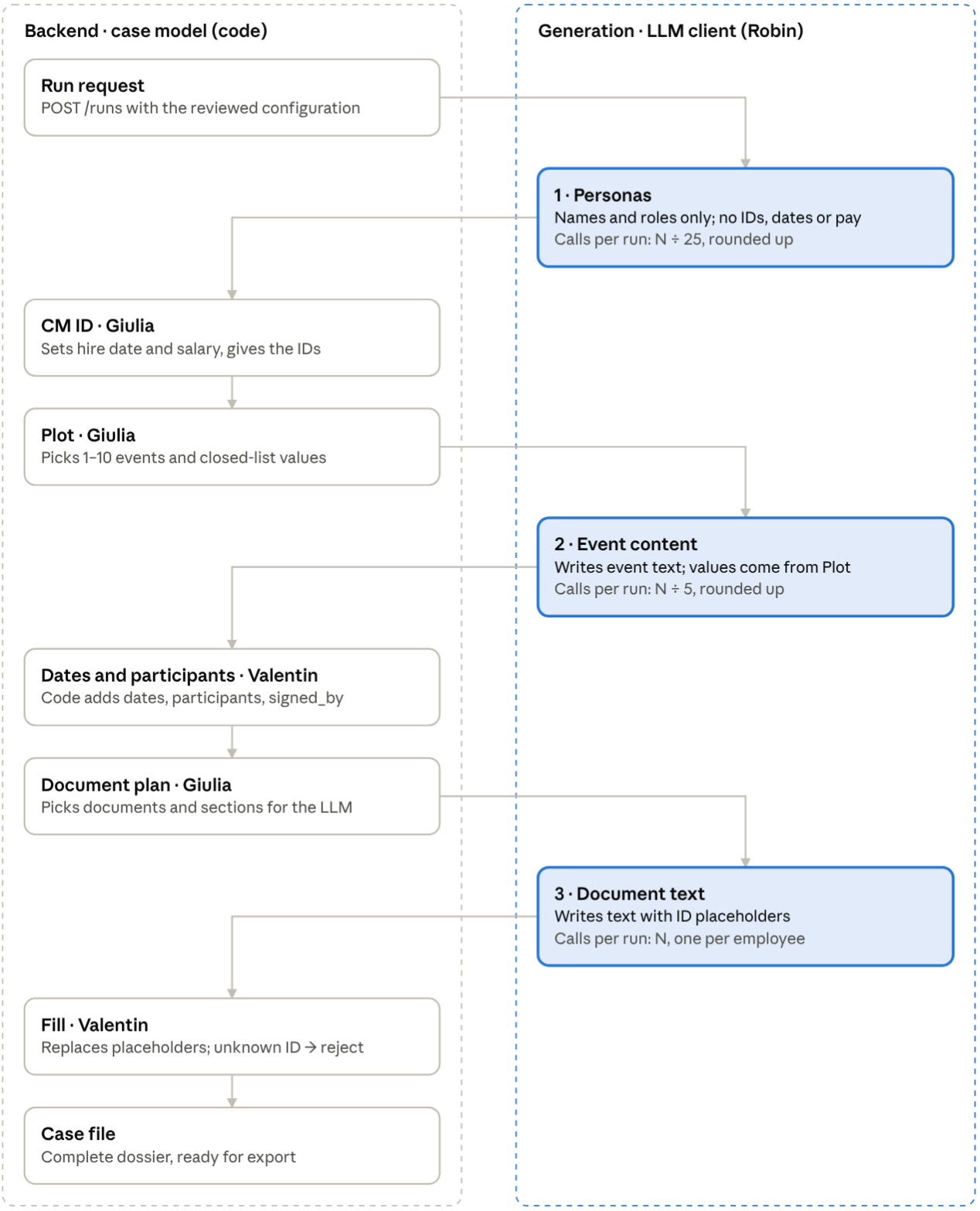
Summary

Generation splits into three LLM calls that run in a fixed order: **Personas**, **Event content** and **Document text**. Each call only proposes JSON. After each one, a case-model step checks it, gives IDs and saves it, and the next prompt is built from that saved state.

For a run of N employees this is N ÷ 25 + N ÷ 5 + N calls, each rounded up: 13 calls for 10 employees, at most 26 with one retry each.

This follows the 30 October meeting and pipeline contract v0.2 (2 November). The contract settles this document's first open points: code also sets hire dates, salaries and closed-list values, and a separate code step plans the documents before Part 3.

Each LLM call proposes; a case-model step checks it before the next



generation in three parts · 3 LLM calls, each followed by a case-model step

The highlighted boxes are the three LLM calls. Every arrow back into the left column is a gate where code checks the answer. N is the number of employees in the run.

Part 1 · Personas

Part 1 proposes the company's employees from the reviewed configuration, in one call per 25 employees. It writes names and roles only: CM ID then sets hire dates and salaries and gives the IDs.

- **Gets:** `RunConfig` from `POST /runs`: company, departments with positions and salary ranges, number of employees, temporary and part-time shares, as-of date, notes. Seed employees in the request (for example the reference 100101) skip Part 1.
- **LLM writes:** per employee, first and last name, department, position, manager or not, contract type, hours per week. Not the hire date or the start salary.
- **Returns:** `PersonasIn`. CM ID turns it into `EmployeesIn`, the shape of `input.employees.schema.json`.
- **Calls:** (N − seed employees) ÷ 25, rounded up (the demo's current limit per call).
- **Checked by:** `complete_as` (shape and types, one retry), then CM ID (department and position must be in the config; anything else becomes a correction or a refusal).
- **Next step:** CM ID (Giulia) sets hire dates and salaries by the template's rules, then gives personnel numbers and the `P` and `ORG` IDs.

The demo's prompt (`personasPrompt` in `prompts.ts`) carries over without its salary and hire-date rules. Its rule that managers have usually been there longest should move into CM ID's hire-date rule. The demo also quietly filled missing employees with offline ones; now a short or malformed answer is retried once, then the run stops with the field named.

Part 2 · Event content

Part 2 writes the text of the events Plot already picked, in one call per 5 employees. Plot also fixes their closed-list values, such as the rating, and the LLM writes text that agrees with them. Dates, amounts, participants and IDs all come from code.

- **Gets:** per employee, the facts CM ID saved (name, position, department, hire date, contract) and Plot's `PlotOutline`: 1–10 slots, each with a type, a slot key from code (`S01`, `S02`, …) and the values Plot already decided.
- **LLM writes:** per slot, the text in the table below, plus a one-line `summary` (the case already has that field).
- **Returns:** `EventContentIn`: per employee number, every slot answered once by echoing its key.
- **Calls:** N ÷ 5, rounded up (the demo's storyline batch size).
- **Checked by:** `complete_as`: no unknown or missing slot, one retry.

- **Next step:** Dates and participants (Valentin) adds dates, participants and `signed_by`, then hands `storylines.json` to `ingest_storylines`. `storylines.json` is no longer an LLM output.

Event type	LLM writes	Code decides
contract_change	Reason (text)	Change kind, new position (Plot); date, new salary, contract type, hours (Dates)
training	Course, duration in days	Date, training provider (Dates)
appraisal	Comment	Rating, year (Plot); date (30 June), signer (Dates)
termination	Nothing beyond the summary	Reason (Plot); date, notice date (Dates)
absence	Nothing: dates only, never a cause	Start and end date (Dates)
medical_check	Nothing	Date, result (Dates)
hire	Not a slot	Everything (CM ID adds it)

The demo's `storylinePrompt` already hands the model some of Plot's choices (who leaves, how many sick-leave periods, which appraisal years). The new prompt goes further: the events and their values are fixed, and the model only writes text for each slot.

Part 3 · Document text

Part 3 writes the free text of each dossier's documents, in one call per employee, using ID placeholders instead of names, dates or amounts. Fill then puts in the facts and rejects any ID the case does not hold.

- **Gets:** one complete case (IDs, dated events, participants, `signed_by`) and its entry in the `DocumentPlan`. A separate code step, Document plan (Giulia), builds that plan after the dates: per document a key (`D01`, `D02`, …), its type, its event and its sections.
- **LLM writes:** the text of each planned section, with placeholders in the agreed form `{{<collection>.<ID>.<field>}}`, for example: *On {{events.EVT-004.date}}, {{personas.P002.name}} reviewed the work of {{personas.P001.name}}.*
- **Returns:** `DocumentTextsIn`: per document key, the text of each section.
- **Calls:** N, one per employee. All of a dossier's texts are written together, so one manager sounds the same across years.

- **Checked by:** `complete_as`, with the placeholder check inside the model. An unknown collection, ID or field fails validation, and so does a name, date or amount typed out instead of a placeholder. That way it gets the one retry with the bad placeholder named, then the run fails.
- **Next step:** Fill (Valentin) resolves every placeholder with the lookup and saves the result in `Document.text`; dates and amounts are formatted only at that point. Then export (Stylianos).

Which documents get LLM text, as accepted in contract v0.2:

Document type	Text from
Performance appraisal	LLM: feedback and summary
Termination letter	LLM: letter body
Contract amendment	LLM: cover letter
Employment contract	Template: fixed legal wording
Training certificate	Template
Payslip	Template and rules
Annual income statement	Template and rules
Medical check	Template: no health details from the LLM
Absence report	Template: dates only, never a cause

Payslips are by far the most numerous documents (one per month), so keeping them templated keeps Part 3 at one call per employee.

All LLM calls in one run

A run of N employees makes N ÷ 25 + N ÷ 5 + N generation calls, each rounded up, plus one or two frontend calls. Every call gets at most one retry.

#	Call	Part, owner	Calls per run	Returns	Code step after it
1	Create analysis	Frontend, Daniel	1 per input, again after each edit	`RunConfig`, reviewed by the user	Known-template check
2	Rule configuration	Frontend, Daniel	0 or 1: only for an unknown template	Template rules	Control layout view

#	Call	Part, owner	Calls per run	Returns	Code step after it
3	Personas	Generation part 1, Robin	(N − seed employees) ÷ 25, rounded up	`PersonasIn`	CM ID
4	Event content	Generation part 2, Robin	N ÷ 5, rounded up	`EventContentIn`	Dates and participants
5	Document text	Generation part 3, Robin	N	`DocumentTextsIn`	Fill

For 10 employees that is 1 + 2 + 10 = 13 generation calls, at most 26 with retries, plus 1–2 frontend calls. The demo caps answers at 300 + 120 tokens per employee for personas and 800 + 2,000 per employee for a storyline batch; document text is not measured yet. Every call now gets its own trace line with tokens and cost (contract §8), so the first runs with the school key give real numbers for Mike's estimate.

The contract between generation and the backend

Generation is three Python functions that the backend's orchestrator calls in order, inside the one service (BE-01). Each returns a validated Pydantic model, so the backend never sees raw model text.

```
propose_personas(config: RunConfig, llm) -> PersonasIn
write_event_content(cases, outline: PlotOutline, llm) -> EventContentIn
write_document_text(case, plan: CaseDocumentPlan, llm) -> DocumentTextsIn
```

Rules that hold at every boundary:

1. Generation never writes an ID. It may only echo a key that code gave it: an employee number, a slot key, a document key.
2. Generation never writes dates, amounts, participants, signers or closed-list values. Code decides them.
3. Each prompt is built from what the case model saved, not from the previous answer.
4. A bad answer gets one retry, then an `LLMError` that names the field. Nothing is half-written.
5. Every call goes through the one LLM client, which records model, tokens, time and retries for Mike's trace (BE-19).
6. Every boundary is a Pydantic model with its JSON Schema exported. An unknown field is refused.

Rule 5 needs one change to the client, now in the contract: `complete()` returns the text and the token usage, and `complete_as` adds the retry count. Each function can be tested offline with `FakeLLM`.

The full contract is `docs/PIPELINE_CONTRACT.md` v0.2 (branch `be-02-pipeline-contract`), with example payloads for every boundary. Worked examples on 100101 for Parts 2 and 3 are still to come, like `EXAMPLE_PAYLOAD.md` for Part 1.

Open questions

- [x] ~~Placeholder syntax: decided `{{<collection>.<ID>.<field>}}` (contract §7).~~
- [x] ~~Which document types get LLM text: the three proposed, accepted.~~
- [x] ~~Start salary and hire date: moved to CM ID.~~
- [x] ~~Closed-list values (rating, termination reason, new position): Plot picks them.~~
- [ ] Keep the demo's batch sizes (25 and 5), then measure with the school key. Robin, Mike
- [ ] A second human review after Fill: mentioned on 30 October, not decided. Daniel
- [ ] Exact shape of `RunConfig` and the review screen that edits it. Daniel, Giulia
- [ ] Part 1 still has the LLM write `contractType`, a closed list, against rule 2. Move contract type and hours to CM ID, so the temporary and part-time shares are hit exactly? Robin, Giulia
- [ ] `durationDays` is the only number the LLM still writes. Move it to Plot, or state its allowed range? Robin, Giulia
- [ ] Part 2 text goes into the case without a placeholder check. Forbid names, numbers and dates in summaries and comments, as the demo's prompt already did? Robin, Giulia